

Designing with RTOS

Task Communication

In a multitasking system, multiple tasks/processes run concurrently (in pseudo parallelism) and each process may or may not interact between. Based on the degree of interaction, the processes /tasks running on an OS are classified as

- **Co-operating Processes:** In the co-operating interaction model one process requires the inputs from other processes to complete its execution.
- **Competing Processes:** The competing processes do not share anything among themselves but they share the system resources. The competing processes compete for the system resources such as file, display device etc

The co-operating processes exchanges information and communicate through

- **Co-operation through sharing:** Exchange data through some shared resources.
- **Co-operation through Communication:** No data is shared between the processes. But they communicate for execution synchronization.

Designing with RTOS

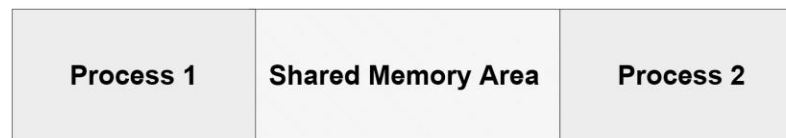
Inter Process (Task) Communication (IPC)

- ✓ IPC refers to the mechanism through which tasks/processes communicate each other
- ✓ IPC is essential for task /process execution co-ordination and synchronization
- ✓ Implementation of IPC mechanism is OS kernel dependent
- ✓ Some important IPC mechanisms adopted by OS kernels are:
 - ✓ Shared Memory
 - ✓ Global Variables
 - ✓ Pipes (Named & Un-named)
 - ✓ Memory mapped Objects
 - ✓ Message Passing
 - ✓ Message Queues
 - ✓ Mailbox
 - ✓ Mail slot
 - ✓ Signals
 - ✓ Remote Procedure Calls (RPC)

Designing with RTOS

IPC – Shared Memory

- ✓ Processes share some area of the memory to communicate among them
- ✓ Information to be communicated by the process is written to the shared memory area
- ✓ Processes which require this information can read the same from the shared memory area
- ✓ Same as the real world concept where a ‘Notice Board’ is used by the college to publish the information for students (The only exception is; only college has the right to modify the information published on the Notice board and students are given ‘Read’ only access. Meaning it is only a one way channel)

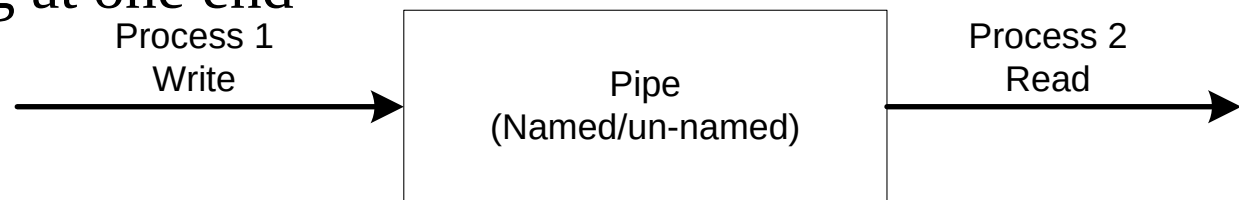


Concept of Shared Memory

Designing with RTOS

IPC – Shared Memory: Pipes

‘Pipe’ is a section of the shared memory used by processes for communicating. Pipes follow the client-server architecture. A process which creates a pipe is known as pipe server and a process which connects to a pipe is known as pipe client. A pipe can be considered as a conduit for information flow and has two conceptual ends. It can be unidirectional, allowing information flow in one direction or bidirectional allowing bi-directional information flow. A unidirectional pipe allows the process connecting at one end of the pipe to write to the pipe and the process connected at the other end of the pipe to read the data, whereas a bi-directional pipe allows both reading and writing at one end



Concept of Shared Memory

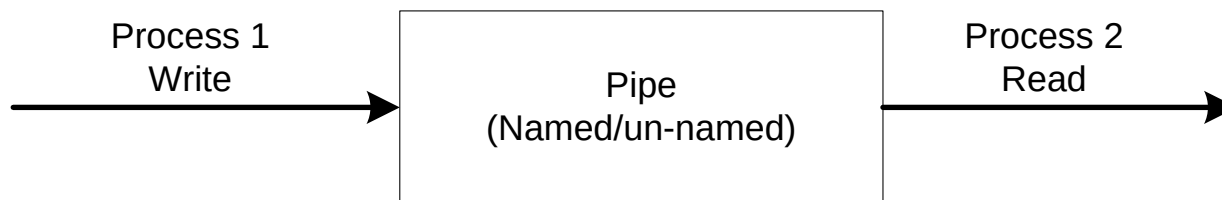
Designing with RTOS

IPC – Shared Memory: Pipes

The implementation of ‘Pipes’ is OS dependent. Microsoft® Windows Desktop Operating Systems support two types of ‘Pipes’ for Inter Process Communication. Namely;

- Anonymous Pipes:** The anonymous pipes are unnamed, unidirectional pipes used for data transfer between two processes.

- Named Pipes:** Named pipe is a named, unidirectional or bi-directional pipe for data exchange between processes. Like anonymous pipes, the process which creates the named pipe is known as pipe server. A process which connects to the named pipe is known as pipe client. With named pipes, any process can act as both client and server allowing point-to-point communication. Named pipes can be used for communicating between processes running on the same machine or between processes running on different machines connected to a network



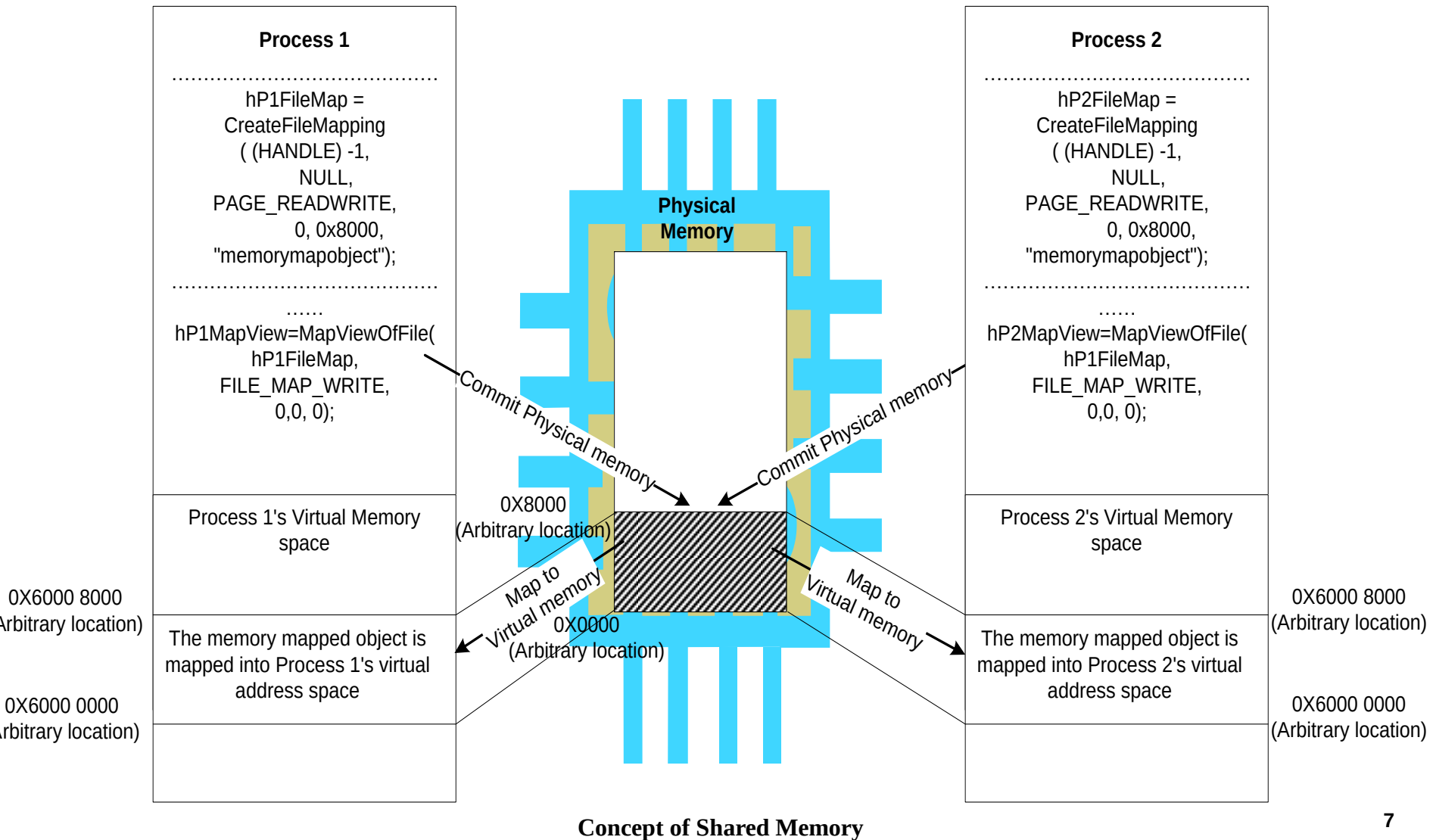
Concept of Shared Memory

Designing with RTOS

IPC – Shared Memory: Memory Mapped Objects

- ✓ A shared memory technique adopted by certain Real Time Operating Systems for allocating a shared block of memory which can be accessed by multiple process simultaneously (of course certain synchronization techniques should be applied to prevent inconsistent results).
- ✓ A mapping object is created and physical storage for it is reserved and committed.
- ✓ A process can map the entire committed physical area or a block of it to its virtual address space
- ✓ All read and write operation to this virtual address space by a process is directed to its committed physical area
- ✓ Any process which wants to share data with other processes can map the physical memory area of the mapped object to its virtual memory space and use it for sharing the data.

Designing with RTOS



Designing with RTOS

IPC – Shared Memory: Memory Mapped Objects

Windows CE 5.0 RTOS uses the memory mapped object based shared memory technique for Inter Process Communication

```
//Create a Mapping from the system memory
```

```
CreateFileMapping (HANDLE hFile, LPSECURITY_ATTRIBUTES lpFileMappingAttributes, DWORD flProtect,  
                  DWORD dwMaximumSizeHigh, DWORD dwMaximumSizeLow, LPCTSTR lpName)
```

lpFileMappingAttributes: The security attributes and it must be NULL.

flProtect : Specify read write access for the shared memory area. PAGE_READONLY → Read only access
PAGE_READWRITE → Read-Write access

dwMaximumSizeHigh : Specifies the higher order 32 bits of the maximum size of the memory mapped object

dwMaximumSizeLow : Specifies the lower order 32 bits of the maximum size of the memory mapped object.

lpName : Points to a null terminated string specifying the name of the memory mapped object. The memory mapped object is created as unnamed object if the parameter *lpName* is NULL.

Designing with RTOS

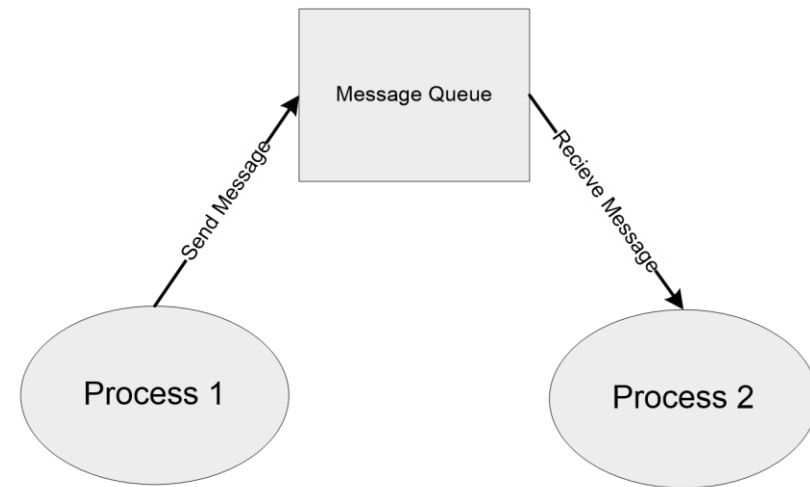
IPC – Message Passing

- ✓ A synchronous/asynchronous information exchange mechanism for Inter Process/ thread Communication
- ✓ Through shared memory lot of data can be shared whereas only limited amount of info/data is passed through message passing
- ✓ Message passing is relatively fast and free from the synchronization overheads compared to shared memory

Designing with RTOS

IPC – Message Passing: Message Queues

- ✓ Process which wants to talk to another process posts the message to a First-In-First-Out (FIFO) queue called 'Message queue', which stores the messages temporarily in a system defined memory object, to pass it to the desired process
- ✓ Messages are sent and received through *send* (Name of the process to which the message is to be sent, message) and *receive* (Name of the process from which the message is to be received, message) methods
- ✓ The messages are exchanged through a message queue
- ✓ The implementation of the message queue, *send* and *receive* methods are OS kernel dependent.

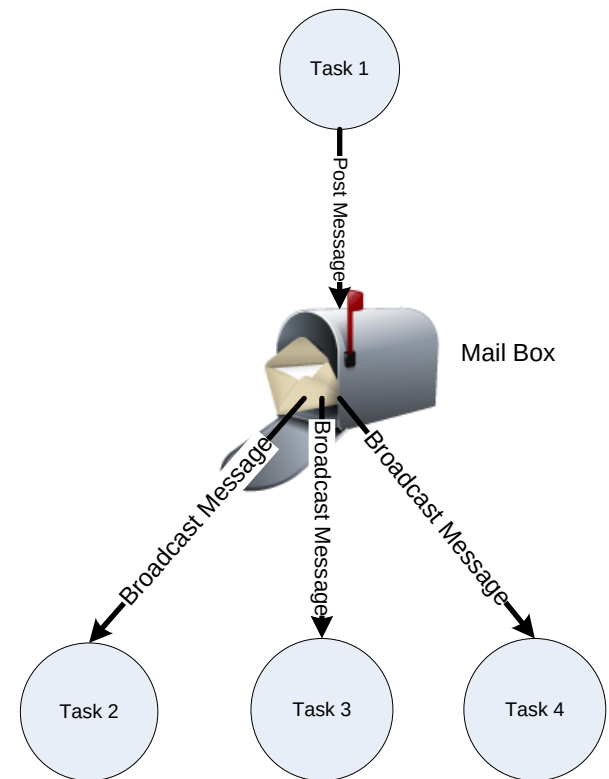


Concept of Message Queue

Designing with RTOS

IPC – Message Passing: Mailbox

- ✓ A special implementation of *message queue*
- ✓ Usually used for one way communication
- ✓ Only a single message is exchanged through *mailbox* whereas ‘message queue’ can be used for exchanging multiple messages
- ✓ One task/process creates the *mailbox* and other tasks/process can subscribe to this *mailbox* for getting message notification
- ✓ The implementation of the *mailbox* is OS kernel dependent
- ✓ The MicroC/OS-II RTOS implements *mailbox* as a mechanism for inter task communication



Concept of Message Queue

Designing with RTOS

IPC – Message Passing: Signal

- ✓ An asynchronous notification mechanism
- ✓ Mainly used for the execution synchronization of tasks process/tasks
- ✓ Signal do not carry any data and are not queued
- ✓ The implementation of *signals* is OS kernel dependent
- ✓ VxWorks RTOS kernel implements ‘signals’ for inter process communication
- ✓ A task/process can create a set of signals and register for it
- ✓ A task or Interrupt Service Routine (ISR) can signal a ‘*signal*’
- ✓ Whenever a specified signal occurs it is handled in a signal handler associated with the signal

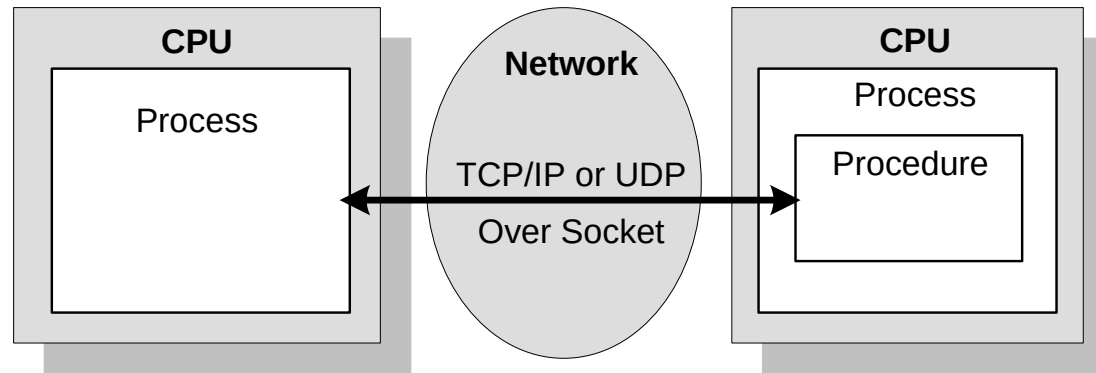
Designing with RTOS

IPC – Remote Procedure Call (RPC)

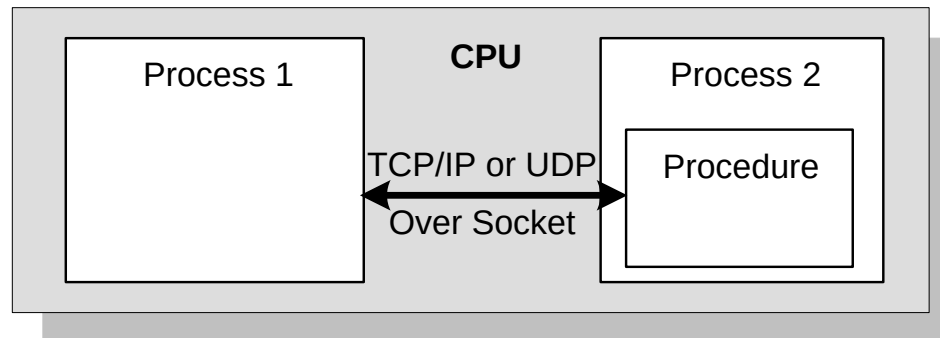
- ✓ The IPC mechanism used by a process to call a procedure of another process running on the same CPU or on a different CPU which is interconnected in a network
- ✓ In the object oriented language terminology RPC is also known as *Remote Invocation* or *Remote Method Invocation (RMI)*
- ✓ RPC is mainly used for distributed applications like client-server applications
- ✓ With RPC it is possible to communicate over a heterogeneous network (i.e. Network where Client and server applications are running on different Operating systems)
- ✓ The CPU/Process containing the procedure which needs to be invoked remotely is known as server
- ✓ The CPU/Process which initiates an RPC request is known as client
- ✓ In order to make the RPC communication compatible across all platforms it should stick on to certain standard formats
- ✓ Interface Definition Language (IDL) defines the interfaces for RPC
- ✓ Microsoft Interface Definition Language (MIDL) is the IDL implementation from Microsoft for all Microsoft platforms
- ✓ The RPC communication can be either Synchronous (Blocking) or Asynchronous (Non-blocking)

Designing with RTOS

IPC – Remote Procedure Call (RPC)



Processes running on different CPUs which are networked



Processes running on same CPU

Concept of Remote Procedure Call (RPC) for IPC

Designing with RTOS

IPC – Remote Procedure Call (RPC)

- ✓ RPC uses *sockets* for implementing communication
- ✓ *Socket is a logical endpoint in a two-way communication link between two applications running on a network.*
- ✓ *A port number is associated with a socket so that the network layer of the communication channel can deliver the data to the designated application*
- ✓ Sockets are of different types namely; Internet sockets (INET), UNIX sockets etc.
- ✓ The INET Socket works on Internet Communication protocol. TCP/IP, UDP etc are the communication protocols used by INET sockets
- ✓ INET sockets are classified into:
 - ✓ Stream Sockets
 - ✓ Datagram Sockets
- ✓ Stream sockets are connection oriented and they use TCP to establish a reliable connection
- ✓ Datagram Sockets rely on UDP for establishing a connection

Multitasking and Task Communication

Learning Summary

- ✓ Processes in a multitasking system can be either *Co-operating* or *Competing*
- ✓ The co-operating processes share data for communicating among the processes through *Inter Process Communication* (IPC), whereas competing processes do not share anything among themselves but they share the system resources like display devices, keyboard etc
- ✓ *Shared memory, message passing* and *Remote Procedure Calls* (RPC) are the important IPC mechanisms through which co-operating processes communicate in an Operating system environment
- ✓ *Pipes* (Named & Un-named), *shared buffers, memory mapped objects* etc are examples of shared memory
- ✓ *Message queues, mailbox, signals* etc are examples of message passing technique
- ✓ The implementation of the IPC mechanism is OS kernel dependent